

Documento de arquitectura.

1. Resumen ejecutivo

Pavia Stationary unifica en una sola página los módulos de catálogo y promociones para que al usuario se le facilite su búsqueda.

Principios:

Principio	Decisión
API	Creamos una API desde 0 para poder guardar información y poder modificarla.
Diseño	Utilizamos CSS, esto para tener una presentación que facilite su uso.

2. Visión y objetivos

2.1. Problema actual

- Dos botones de inicio
- Mapa de ubicaciones sin funcionalidad

2.2. Objetivos

2.2.1. Estética armoniosa: El diseño de la página no utiliza colores neutrales y presenta los productos de una manera ordenada.

2.2.2. Forma de usar sencilla: Con ayuda de la presentación, ayuda a que el cliente pueda utilizar la web sin ningún problema.

2.2.3. Presentación de tienda: Nuestra página tiene como objetivo el presentar el inventario de nuestra papelería, así como las promociones que tiene.

2.2.4. Inicio de sesión privada: Al iniciar sesión en nuestra página, guarda la información de forma privada en una base de datos.

2.3. Usuarios

Rol	Cantidad	Necesidad
Desarrolladores	Varios	Codigo, mantenimiento
Diseño	3	Actualización constante y código.
Documentación	1	Conocimiento básico sobre los procesos hechos en el programa.

3. Inventario

3.1. Papelería

3.1.1. src

3.1.1.1. App.jsx: Es el componente principal, brindando la mayoría de las funciones del programa.

3.1.1.2. App.css: Es específicamente para aplicar diseño en el apartado de app.

3.1.1.3. Index.css: Es específicamente para aplicar el diseño pero en toda la página.

3.1.1.4. Main.jsx: Es el punto de entrada.

3.1.2. public

Son archivos estáticos.

3.1.3. index.html

Es la base del HTML.

3.1.4. package.json

Almacenan scripts y dependencias.

3.1.5. vite.config.js

Es solamente para la configuración de Vite.

3.2. README.md

4. Arquitectura de software

- CSS: Lo utilizamos a la hora de crear un diseño para todo el programa, utilizamos varios comandos para crear efectos visuales y que todo se vea más organizado. El CSS se iba creando mientras el JS avanzaba.
- React: Con el código de JS, utilizamos React para agilizar la programación, reutilizando algunos elementos para no tener un código muy grande. Lo utilizamos ya que todo el Javascript estaba terminado y mientras se creaba el código.
- Javascript: Lo utilizamos para crear todas las funcionalidades del programa, fue con lo que iniciamos.

5. API y Dashboard

5.1. API

5.1.1 GET: /api/ofertas, /api/productos, /api/sucursales

5.1.2 PUT: /api/ofertas/:id

5.1.3. DELETE: /api/ofertas/:id

5.1.4. POST: /api/ofertas

5.2. Dashboard

Petición	Imagen
GET	<pre>return (<div className="pagina-ofertas"> <h2>Lista de Productos</h2> <div className="grid-catalogo"> {productos.map((producto) => (<CardCatalogo key={producto.id} imagen={producto.imagen} titulo={producto.nombre} precioOriginal={producto.precio} mostrarBoton={false} />)} </div> </div>)</pre>
PUT	<pre><Link to="/admin/ofertas/agregar"> <BotonPrincipal texto="Agregar Ofertas" variante="oscuro" /> </Link></pre>
DELETE	<pre>// Confirmar eliminación const confirmarEliminar = async () => { if (!idEliminar) return await fetch(`\${API}/ofertas/\${idEliminar}`, { method: 'DELETE' }) setOfertas(ofertas.filter((o) => o.id !== idEliminar)) setIdEliminar(null) }</pre>
POST	<pre><BotonPrincipal texto="CAMBIOS" variante="gris" onClick={() => navegar('/admin/ofertas/editar/\${oferta.id}')} /> </>)) (modos === 'modificar' && (</></pre>

6. Despliegue

El código se muestra completo con todos sus apartados en GitHub.

7. Fase de implementación

7.1. Se inició con el FrontEnd y BackEnd

7.2. Se utilizó REACT y JSX para implementarlo al código y agilizarlo

7.3. Al finalizar de programar, se documentó todo

Andres	Hanna	Eduardo	Keren	Victoria	Sofia
FullStack Developer	Project Manager & FrontEnd Developer	BackEnd Developer	BackEnd Developer	FrontEnd Developer	Documentación

8. Decisiones técnicas

8.1. Pantalla de Login: Se utilizará para tener una mejor organización a la hora de ver la información sobre los administradores.

8.2. Pantallas de catálogo: La creamos para que el usuario pueda ver el producto que le interese junto a su información, consta de diferentes categorías la

cual facilita el buscar el producto más rápido. Para esta parte cabe aclarar que existen múltiples pantallas para mostrar todos los productos.

8.3. Pantalla de promociones: Sirve para que se muestren solo promociones de nuestra papelería. Cuenta con una sola pantalla especialmente para esto.

Con estas decisiones tomadas se espera que fuera una página funcional el cual sea atractiva para el usuario y que sea fácil de usar, esto para que tenga más información sobre nuestros productos y promociones.

9. Riesgos

- Login inseguro.
- Facilidad para acceder al módulo de administración.

10. Glosario

- **JSX:** Es una extensión de sintaxis para JavaScript que permite escribir código similar a HTML directamente dentro de archivos JavaScript. Se utiliza principalmente en React para describir cómo debe verse la interfaz de usuario, combinando la estructura visual con la lógica funcional.
- **API (Interfaz de Programación de Aplicaciones):** Es un conjunto de reglas y protocolos que permite que diferentes programas de software se comuniquen entre sí. Actúa como un puente o intermediario, permitiendo que una aplicación solicite datos o servicios a otra sin necesidad de conocer su código interno.
- **Petición:** Es un mensaje que envía un programa o cliente a un servidor para solicitar un recurso, pedir información o ejecutar una acción. Existen 4 tipos de peticiones:
 1. GET: Mostrar información
 2. POST: Modificar información existente.
 3. DELETE: Eliminar información
 4. PUT: Agregar nueva información.